PLC Structured Text Coding Agent

Implementation and a Four-Condition CODESYS Benchmark

Generative AI Course Final Project

27 October 2026

Abstract

This project implements a lightweight agent harness that generates Structured Text, invokes the actual CODESYS compiler, and applies bounded repairs using compiler diagnostics. We compare it with the unmodified smolagents ToolCallingAgent using two available relay model identifiers on all 20 official tasks and 12 independently constructed tasks. The dataset contains explicit scan semantics, 207 LTL properties and 81 fixed functional cases. The experiment preserves failed requests, every candidate, compiler evidence and real IDE-simulation traces. Compilation and finite functional correctness are evaluated separately. Results, coverage and limitations are reported from manifest-selected records, without treating Python reference checks or unexecuted cases as PLC successes.

1 Problem and engineering objective

Structured Text (ST) is used to express retained state, sequencing and arithmetic in programmable controllers. A plausible code response is not enough: the source must fit a real project, compile under the installed toolchain, and behave correctly when called across scans. Our objective is an explainable generation–compilation–repair pipeline with evidence that can be independently inspected.

The submission includes the agent implementation, official and self-built task definitions, LTL specifications, fixed tests, isolated CODESYS projects, raw model and compiler records, analysis scripts, this report, an editable presentation and a short narrated explanation. The comparison does not assume that our harness should win. It measures the two complete agent workflows under shared controls and preserves their unsuccessful runs.

2 Environment and reproducible execution

The host is Windows 11, build 26200. Application code runs in the existing Conda environment `Project1`, using Python 3.12.7 and Requests 2.32.5. NumPy 1.26.4, pandas 2.3.3 and Matplotlib 3.10.6 support analysis. Existing PyTorch 2.2.2 was retained; the experiment requires no local GPU inference or model weights. Lightweight baseline dependencies were added without replacing the ML environment.

CODESYS Development System 3.5.22.40 is installed on drive D. The exact profile is `CODESYS V3.5 SP22 Patch 4`, with ScriptEngine 4.2.0. Its embedded scripting interpreter is separate from Project1. The benchmark template targets the installed Control Win V3 x64 device. The original vendor template required an explicit target/library upgrade; the resulting working template was frozen before the formal campaign and its hash is recorded in compiler evidence. IDE simulation is used for runtime testing; no physical PLC is involved.

The common configuration permits an initial build and at most two repairs, a 180-second compiler limit, a 180-second model-request limit, a 900-second task deadline, medium reasoning effort and `max_output_tokens=8192`. Request and compiler limits are clipped to remaining task time where implemented. Actual elapsed time remains the measurement, rather than an assumption of identical framework overhead. All CODESYS operations share a bounded file lock.

API credentials are loaded at runtime from environment variables or a private local configuration outside the project. Request headers are not serialized; local key files, environments, caches and installers are excluded from the release ZIP. Reproduction instructions are in `docs/reproducibility.md`; executable settings are in `configs/runtime.json`. On another installation, these machine-specific paths must be updated.

3 Dataset and formal specifications

3.1 Sources, coverage and construction

All 20 instructor-provided questions are included as PUB001–PUB020. The original JSON requirements are preserved verbatim, and their titles, difficulty and interface tokens were cross-checked with the supplied 25-page question-bank PDF. Source binaries, extracted text, hashes and the alignment audit remain under `requirements/source/`. Recorded interpretations supplement the official statements rather than silently replacing them.

Twelve self-built tasks, SELF001–SELF012, cover guarded motor retention, reversal interlocks, restart inhibition, pallet indexing, traffic phases, tank hysteresis, alternating pumps, consecutive-scan qualification, edge counting, alarm acknowledgment, actuator handshakes and bounded fault retries. Each records independent synthesis/adaptation provenance and practical motivation. They were constructed and reviewed by the coding assistant, without external expert validation. They represent different state and priority problems rather than variations of a single latch.

Table 1: Frozen dataset. Case and scan counts are planned tests, not observed passes.

Quantity	Count
Official public tasks	20
Self-built tasks	12
Easy / medium / hard tasks	10 / 14 / 8
LTL properties	207
Fixed functional cases	81
Planned scan observations	576

Each task defines natural-language requirements, scenario, typed interface, retained initial state, scan semantics, safety/priority rules, assumptions, atomic propositions, LTL formulas, fixed inputs and expected outputs. The dataset digest is frozen in the campaign manifest and each selected result includes a task hash. Models receive the full tests, so this is specification-following evaluation, not a hidden-test generalization study.

3.2 Scan convention and meaningful LTL

For invocation k, the caller samples $I[k]$, executes the FB from retained pre-state $q[k]$, and observes $O[k]$ at the end of that same invocation. The updated state is $q[k+1]$. Step zero executes normally. Each case starts with a fresh FB instance, and initial conditions refer to its pre-state.

For the guarded motor SELF001, two safety and startup properties are

$$\mathbf{G}\big((Stop \vee \neg GuardClosed) \rightarrow \neg Run\big),$$
$$\mathbf{G}\big((Start \wedge \neg Stop \wedge GuardClosed) \rightarrow Run\big).$$

They specify same-scan stop priority. Retention is separately stated against the pre-state proposition `qRun`. The assignment's motor example uses $\mathbf{X}\neg Motor$ and $\mathbf{X}Motor$, with a table showing the state in the following row. That pre-scan Moore convention is compatible with a delayed row label, but applying its $\mathbf{X}$ directly to our end-of-scan observations would introduce an incorrect delay. The mapping is documented explicitly in `docs/ltl_semantics.md`.

Properties also cover mutual exclusion, state transitions, faults and conditional liveness. Numeric predicates are defined in the atomic-proposition dictionary. Liveness is conditional on environmental assumptions such as continuing enable and recurring ticks. A fault may legitimately wait forever for reset, so a weak-until expansion is used instead of falsely requiring eventual reset. These formulas are specification artifacts; the project does not claim that all 207 have been model checked.

The independent Python reference models passed 24 self-task cases with 142 scan observations. Selected finite exploration checked 44 safety predicates over 96 states and 5,152 transitions. These checks support expected-sequence consistency only. Their explicit `PYTHON_REFERENCE_ONLY` evidence is excluded from generated-ST functional rates.

4 Self-developed agent harness

The harness is a small deterministic controller around remote model generation and an external compiler. Requirement analysis extracts the interface, initial state, scan convention, priorities, assumptions and temporal specifications into a retained analysis artifact. The generator returns structured JSON with an `FB_Task` declaration, implementation, and optional helper types or POUs.

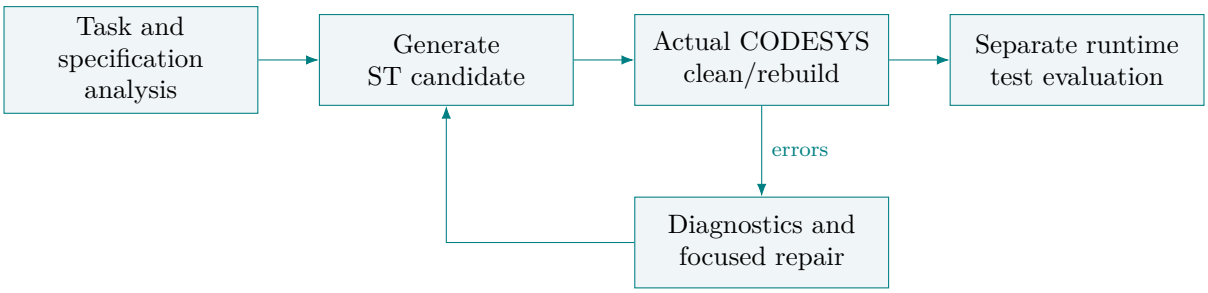


Figure 1: Harness control flow. The feedback loop has a maximum of two repairs.

After a failed build, the analyzer selects and deduplicates relevant diagnostics. A focused repair request includes the original task, current candidate, errors and compact history. The harness stops on compilation success, exhausted budget, timeout or infrastructure failure. Every version and request is retained. A successful build initially yields overall `UNVERIFIED`; only actual functional evaluation can establish a finite-test pass.

The implementation is more than a single prompt: it owns the source contract, project invocation, evidence validation, diagnostic feedback, bounded retries, termination and trace evaluation. It remains simple enough to explain without attributing the experiment to a complex internal multi-agent system.

5 Actual CODESYS compilation

The reusable `compile_st(task,source,workspace)` backend creates a fresh template copy and launches CODESYS with `--runscript`, `--scriptargs` and `--noUI`. The internal script inserts source through editor APIs, reads it back, forces the FB into the build, clears messages, cleans and rebuilds. Error and FatalError messages determine compiler rejection. Successful evidence also requires a compiled POU signature/CRC, matching request nonce and source digest, and normal process completion. The Windows launcher quotes option values in the form expected by the installed executable.

Table 2: Actual backend preflight. All three processes returned exit code zero.

Fixture	Errors	Backend result
Known valid motor latch	0	PASS
Undefined identifier deliberately inserted	2	COMPILE_FAIL
Corrected source	0	PASS

Listing 1: Actual invalid-fixture diagnostic excerpt (text evidence, not an IDE screenshot).

```
Identifier 'ThisIdentifierDoesNotExist' not defined
Compile complete -- 2 errors, 0 warnings
Process exit code: 0
```

The preflight demonstrates why process exit status alone is insufficient. It validates the shared backend; its deliberately supplied correction is not counted as model-driven repair. Isolated projects, source, full logs and evidence for the three runs are linked from `results/summary/compiler_preflight.json`. The official scripting interfaces document script startup, rebuilding and diagnostic access [1, 2, 3].

6 Baseline, models and experimental controls

The existing-agent baseline is the unmodified `smolagents.ToolCallingAgent` version 1.26.0. A model subclass translates its messages/tool calls to the relay's Responses streaming API, and a compiler Tool exposes the same backend and enforces the common three-build budget. The framework owns its ReAct prompt, memory, tool selection, execution loop and final-answer action [4]. It receives no shell or general filesystem tool and starts fresh for every task.

Codex CLI was initially preferred. Actual setup conversations reached the relay, but its local noninteractive tool policy blocked shell/MCP execution. No policy was bypassed. The baseline was explicitly changed before the full campaign, and the old manifest, setup failures and revision explanation are retained. They are not formal smolagents rows.

Both selected IDs, `deepseek-v4.1-flash` and `kimi-k3.1-pro`, appeared in the provider catalog and completed a real streamed function-call/result round trip at medium reasoning effort. Both then generated candidates that the baseline and harness actually compiled. This establishes tested protocol compatibility. The relay labels do not independently prove upstream vendor/model identity; prices and context limits remain unknown.

Task JSON, expected tests, compiler version/template, build budget and request settings are shared. Internal prompts and context policies intentionally differ. The baseline may make a fourth model call to produce its normal `final_answer`, whereas the harness normally stops immediately after a successful build. Tool event counts therefore have framework-specific units. Each condition/task is run once; no statistical superiority or causal policy ablation is inferred.

Table 3: The four conditions, each with the same 32 tasks.

Condition	Agent framework	Exact relay model ID
E1	smolagents 1.26.0	`deepseek-v4.1-flash`
E2	smolagents 1.26.0	`kimi-k3.1-pro`
E3	Self-developed harness	`deepseek-v4.1-flash`
E4	Self-developed harness	`kimi-k3.1-pro`

Runs are isolated in per-task directories. The runner resumes only `NOT_RUN` entries and preserves earlier attempts when an explicit rerun changes a manifest pointer. One early E3/PUB001 TLS setup failure was rerun once and retained in history; other formal failures were not repeatedly retried to select the best answer. Generation interrupted between tasks was resumed without altering the dataset, prompts or budgets.

7 Functional evaluation and metric definitions

The automatic evaluator executes the generated ST in the actual CODESYS IDE simulator. It enables and verifies simulation mode before login. A native ST `CASE` sequencer assigns fixed inputs, calls the candidate exactly once per real simulator task scan, and records actual sampled inputs and outputs in typed arrays. Each case uses a fresh instance. The host stops after the exact expected invocation count and obtains outputs through the CODESYS monitoring API. Expected outputs never appear in the executed caller.

The current validated adapter covers 21 tasks: all twelve self-built tasks and PUB001, PUB002, PUB003, PUB009, PUB011, PUB014, PUB015, PUB019 and PUB020. They contain 52 cases and 359 scans. These are eligibility counts, not passing-test counts. Physical elapsed-time tests, TIME/LTIME, IN_OUT storage, arrays, structures and VARIANT bindings remain unsupported. The project supplies fixed manual procedures for those cases and does not claim compressed wall time or unexecuted correctness.

Trace evaluation requires task and source hashes, fixed input agreement, typed expected-output comparisons and hashed supporting evidence. Time-based imports require measured intervals within a registered tolerance. Missing or invalid evidence cannot establish a pass.

First-pass and final compilation rates divide successful tasks by all 32 planned tasks in a condition. Compiler coverage is reported separately, so a transport failure is not described as a CODESYS code rejection. A case passes only if all its constrained outputs match. Case pass rate divides passed cases by evaluated cases; task pass rate requires all fixed cases to be evaluated and passing, with task coverage also shown. Missing observations remain unknown. There is a difference between all 128 end-to-end attempts being executed and all 128 candidates reaching the compiler.

Token totals use reported provider usage only. Missing usage remains unknown, including failed streams. Because the harness's stored call counter increments after successful responses, independent analysis counts saved `llm_*/request.json` files for both workflows to obtain attempted requests. No monetary cost is inferred from incomplete usage and unavailable prices.

8 Four-condition benchmark results

The following tables are generated directly from the manifest-selected records by `scripts/analyze_results.py`. All conditions retain their planned denominator of 32. An infrastructure failure is an unsuccessful end-to-end task attempt but is not recorded as an observed compiler rejection. Repair entries show recovered initial failures over observed initial failures; a zero denominator means no measured opportunity to estimate repair effectiveness.

Table 4: Compilation outcomes. All rates use 32 planned tasks per condition; infrastructure failures stay in the denominator.

Condition	Attempted	First pass	Final pass	Compiler evaluated	Repairs
E1	32/32	27/32 (84.38%)	29/32 (90.62%)	29/32	2/2
E2	32/32	27/32 (84.38%)	27/32 (84.38%)	28/32	0/1
E3	32/32	26/32 (81.25%)	26/32 (81.25%)	26/32	0/0
E4	32/32	30/32 (93.75%)	31/32 (96.88%)	31/32	1/1

Table 5: Actual CODESYS functional results. Pass rates are conditional on evaluation; planned totals are 81 cases and 32 tasks per condition.

Condition	Cases passed	Case rate	Case coverage	Tasks passed	Task rate	Task coverage
E1	42/45	93.33%	45/81	17/19	89.47%	19/32
E2	46/48	95.83%	48/81	18/19	94.74%	19/32
E3	38/38	100.00%	38/81	18/18	100.00%	18/32
E4	50/52	96.15%	52/81	20/21	95.24%	21/32

8.1 Observed framework and model differences

E4 achieves the highest final compilation rate in this run, 31/32 (96.88%), while E1 reaches 29/32 (90.62%). The direction of the framework difference changes with the relay model: the harness trails the baseline by three final compilations with model A, and leads by four with model B. These are descriptive paired-task outcomes, not evidence of a general framework advantage. Fourteen provider/transport failures and one observed final compiler failure remain in the planned denominators.

Public tasks are harder to complete end to end than the self-built set in most conditions. Final public compilation counts are 17/20, 17/20, 14/20 and 19/20 for E1–E4; self-built counts are 12/12, 10/12, 12/12 and 12/12. All easy tasks compile in E3/E4, but only 5/8 hard tasks compile in E3. Failed transports contribute to these differences, so difficulty and code-generation ability cannot be isolated from this single campaign.

The repair loop recovers three actual initial compiler failures: two in baseline E1 and one in harness E4. E2 stops after its one failed PUB018 build. E3 has no observed compiler rejection to repair; its six unsuccessful tasks terminate in the relay before a compiler observation. A zero repair denominator therefore provides no evidence that a repair policy is ineffective.

8.2 What the finite runtime tests establish

Across four conditions, the simulator evaluates 183 of the 324 planned case instances and passes 176. Eligible scalar cases are executed whenever the selected candidate compiled; unsupported timer/composite bindings remain unverified. E3's 38/38 case result has the smallest execution

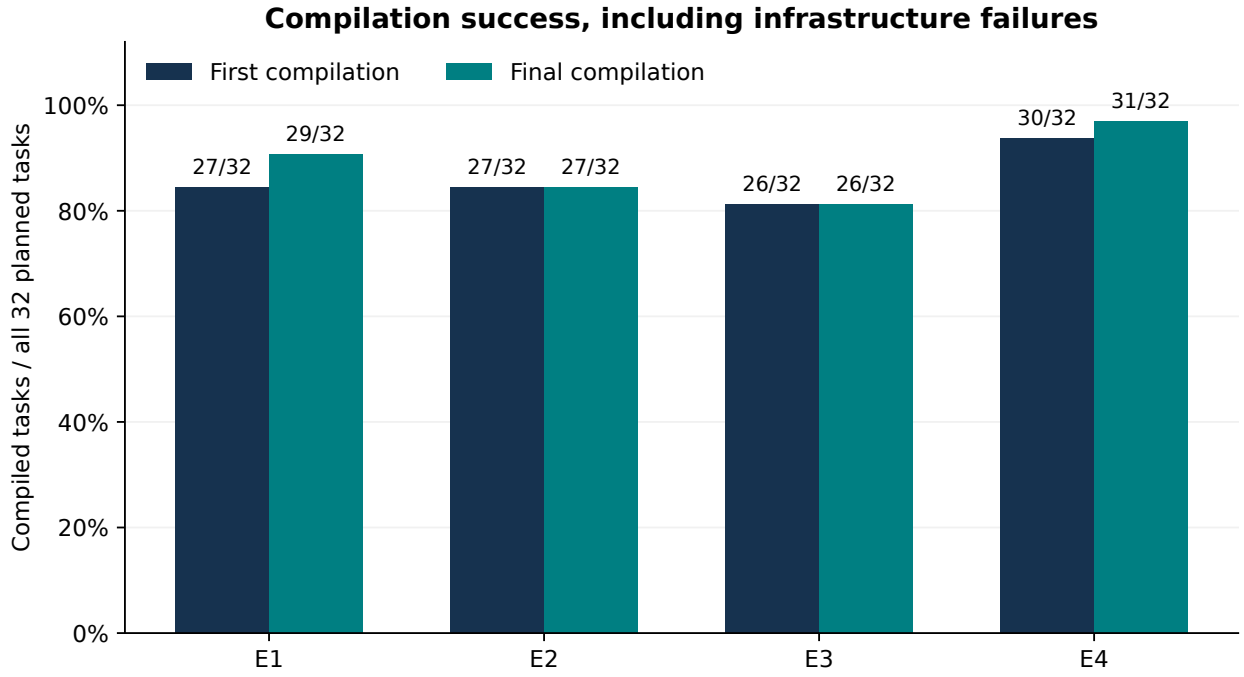


Figure 2: Actual first-pass and final compilation rates. All 32 planned tasks remain in each denominator.

coverage (38/81), and cannot be read as complete correctness or a directly comparable 100% benchmark success rate.

PUB014 fails both cases in E1, E2 and E4. For example, E1 returns zero for the actual input `1.2.3.4`, whose fixed expected network-order value is 16909060. The acquired input traces match the test inputs, and the exact compiled source hash is checked before simulation. The observation establishes incorrect candidate behavior on those scans; it does not by itself identify the source-level cause. E1 also fails SELF004's jam case at zero-based scan 5. The remaining 176 cases pass their fixed scan assertions. None of these finite traces proves every LTL property over unbounded execution.

The repaired E4/SELF012 candidate passes both of its actual runtime cases. This links one successful harness repair to finite behavior checks, while the E1/PUB014 example shows why a recovered build must still be tested. Detailed failed scan indices and trace paths are included in `results/analysis/functional_failures.csv`.

E4's mean agent time (70.0 seconds) exceeds its median (49.3 seconds), indicating a longer upper tail. Baseline E2 makes more saved request attempts (77) than E1 (65), whereas the harness makes 32 and 33. Framework-internal final-answer and planning interactions differ, and incomplete usage records preclude a complete token-cost comparison. No monetary efficiency claim is made.

8.3 Functional correctness and coverage

The case rate is conditional on actual evaluation and must be read together with coverage. Each condition plans 81 cases. The current automatic adapter is eligible for 52 of them, and generation/compilation failures can further reduce execution coverage. All remaining cases have fixed manual procedures but cannot be counted as passes without actual trace acquisition. The scalar simulator tests exercise the exact final compiled source; they do not substitute the Python reference implementation.

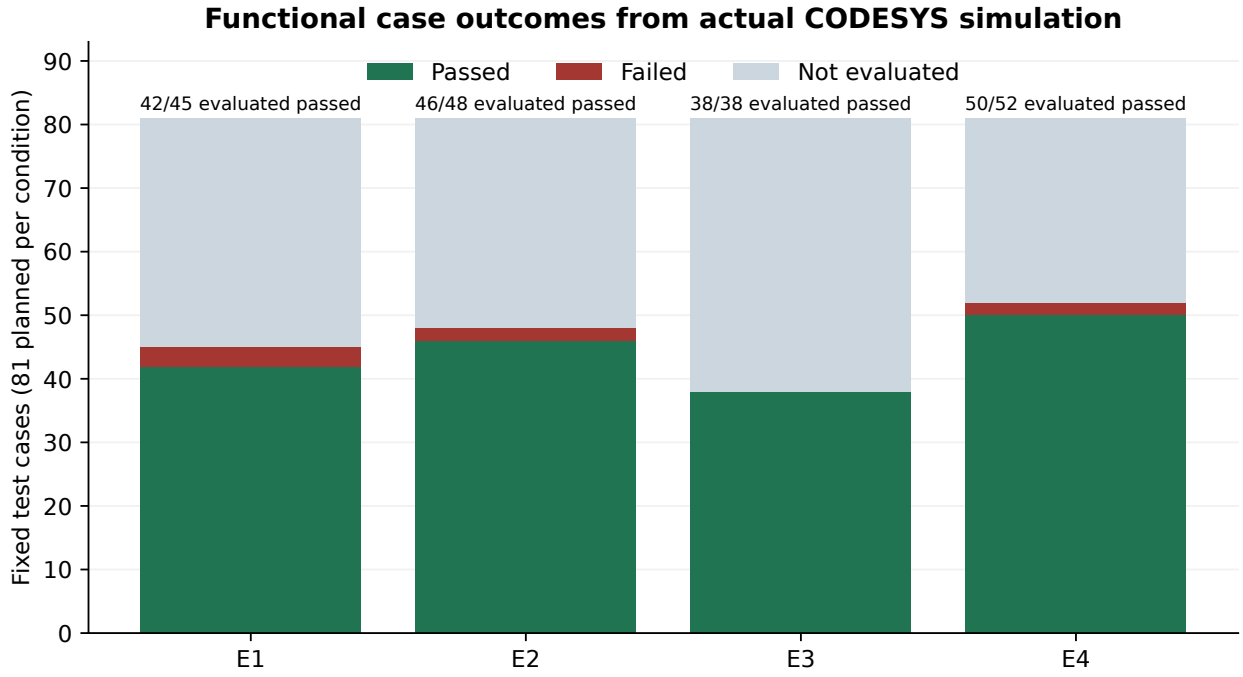


Figure 3: Finite functional-case outcomes and observed coverage against all 81 planned cases.

8.4 Resources and reliability

Elapsed time includes the agent's model interactions, compiler work and controller overhead, but excludes the separately run functional campaign. Request-file counts measure attempted calls consistently across frameworks. Reported input/output totals remain incomplete whenever a request has no usage record; they are not estimates for missing calls.

Table 6: Original agent-run resources. Times include failed runs and compiler calls. Token counts sum observed provider reports only; unknown usage is not zero.

Condition	Mean/median seconds	Requests	Usage coverage	Observed input tokens	Observed output tokens	Compiler calls
E1	41.6/34.4	65	62/65	369,513	88,844	31
E2	50.0/50.9	77	73/77	458,347	26,742	28
E3	47.9/41.8	32	26/32	89,846	85,898	26
E4	70.0/49.3	33	32/33	127,912	47,125	32

Complete per-task, source-group, difficulty, infrastructure-error and functional-failure breakdowns are provided in `results/analysis/`. The raw records are the authoritative source. Aggregate status is not used to overwrite whether a build actually succeeded.



Figure 4: Observed generation-agent wall time per task. One run per condition/task limits interpretation.

9 Case studies from actual records

9.1 Compiler-guided repair: E1/PUB014

PUB014 asks for strict dotted-decimal IPv4 parsing into a network-order UDINT, returning zero for malformed input. The E1 baseline's first candidate declared a local integer `len` and used `len:=LEN(ipText)`. CODESYS reported two errors:

```
Cannot convert type 'Unknown type: 'LEN(ipText)'' to type 'INT'
Program name, function or function block instance expected instead of 'LEN'
```

The diagnostic links to the local declaration. A likely explanation is that `len` hides the case-insensitive library function; this is an interpretation of the actual declaration and messages. The next model candidate removed the local variable and replaced the length-based loop with a bounded scan terminating at the string's zero byte:

```
(* First candidate; local len : INT *)
len := LEN(ipText);
FOR i := 1 TO len DO
    c := ipText[i];
    (* segment validation *)
END_FOR

(* Repaired candidate; local len removed *)
i := 1;
WHILE (i <= 254) AND ok DO
    c := ipText[i];
    IF c = 0 THEN EXIT; END_IF
    (* segment validation *)
    i := i + 1;
END_WHILE
```

The second actual build had zero errors and warnings after one repair. The source digests changed from `b6638e0b22e5635b...` to `3b43179efdf7cdb5...`, and both builds have separate nonces and project evidence. Both processes exited zero. This is evidence of compiler-feedback repair in the existing-agent baseline, not a repair success attributable to our harness. Both subsequent runtime cases fail (13/19 scan observations match), including a zero result for input `1.2.3.4`. Compiler-guided repair therefore solved a build problem without establishing the required parser behavior.

9.2 Compilation does not establish task fidelity: E1/PUB018

The official FIFO interface has caller-owned `item:VARIANT` and `buffer:VARIANT` storage and requires generic element types and array bounds. E1's initial source produced 38 CODESYS errors, including unknown types and invalid pointer/conversion expressions. After feedback, it added this helper DUT and rewrote byte-copy loops:

```
TYPE VARIANT :
STRUCT
    pData : POINTER TO BYTE;
    udiSize : UDINT;
END_STRUCT
END_TYPE
```

The repaired candidate compiled with zero errors and warnings. That is a build success under the stated metric. However, a caller must now supply a custom pointer-and-size structure. Preserving the word `VARIANT` does not establish native generic compatibility, array-bound behavior or correct FIFO execution. The agent disclosed this substitution in its final answer. This task remains functionally unverified because the current adapter cannot validate its IN_OUT/composite binding. We do not describe it as a verified generic FIFO.

9.3 Early agent termination: E2/PUB018

The Kimi-route baseline made one build attempt, receiving two unknown-VARIANT type errors. It then called `final_answer` and declined to substitute an adapter. No repairs occurred. Three build attempts were available, so this was early agent termination, not budget exhaustion. The candidate also omitted the full data-copy implementation. Its `COMPILE_FAIL` result cannot be interpreted as an otherwise complete implementation failing only because of platform portability.

9.4 A successful repair by our harness: E4/SELF012

The final generation run provides a separate harness example. SELF012 specifies bounded automatic fault retries with a recovery dwell. The first candidate introduced enum literal `LOCKED` and required BOOL output `Locked`. CODESYS reported three errors: two INT/BOOL comparison errors at implementation lines 5 and 40, and a BOOL-to-`E_TASKSTATE` conversion error at line 23. This is consistent with case-insensitive identifier collision in the candidate's expressions.

```
(* Initial helper enum and assignment *)
(IDLE, RUNNING, WAIT, LOCKED)
State := LOCKED;

(* After compiler-guided repair *)
(TASK_IDLE, TASK_RUNNING, TASK_WAIT, TASK_LOCKED)
State := TASK_LOCKED;
(* Required output remains Locked : BOOL *)
```

The repair prefixes all enum literals and also adds an explicit locked-state CASE branch. The next real build reports zero errors and warnings after one repair. Candidate hashes change from `6337eb8fb5e63ea6...` to `7c914ea2dd50a9c4...`. This third recovered build belongs to our harness, while the preceding two repair successes belong to baseline E1. Both subsequent simulator cases pass. This demonstrates the implemented feedback loop and finite behavior checks without claiming an ablation or unbounded correctness.

9.5 Transport failure: E3/PUB010

This run saved a real model request, but the stream ended with an upstream-closed-before-terminal-event error. No complete candidate was accepted and no CODESYS build occurred. It is `TOOL_ERROR`, not a compiler rejection. The task remains in the planned denominator, while compiler coverage shows the missing evaluation.

The saved request also establishes one attempted call even though the harness's stored counter is zero. Independent request-file accounting corrects the comparison without silently rewriting raw records. Missing usage does not prove zero upstream computation or zero charge, and the interruption alone does not prove its cause. Full case links, source versions and diagnostics are in `report/case_studies.md`.

10 Limitations and reflection

The most important distinction is between an executed agent run, a compiled candidate and a behaviorally checked controller. A relay outage prevents compilation without demonstrating a syntax error. A successful build can still change an interface or fail a scan trace. Even complete fixed-test success does not prove every input sequence or unbounded LTL liveness.

The automatic runtime adapter has a deliberate coverage boundary: scalar, untimed interfaces. It does not validate physical timing, caller-owned composite storage or the generic FIFO contract. Those tasks retain manual procedures and expected results. The current project does not claim a completed manual campaign, PLCVerif integration, certification, physical PLC deployment or additional-platform support. The reference exploration concerns selected Python-model predicates only.

One observation per task/condition is inadequate for separating model randomness, relay stability and framework effects. The tasks mix source groups and difficulty levels, and only three compile failures were recovered in the observed campaign. Differences in internal prompts, context and the baseline's final-answer request remain framework effects. Both agents see test expectations. Claims about hidden-test generalization or universal superiority would go beyond this design.

The exact identifiers are provider routes. Upstream identities, context sizes and prices could not be independently established. Reported token totals are partial observations, not complete billing estimates. Raw interruption events and usage gaps are retained. A future uniform retry policy or repeated campaign would need to be declared before selecting replacement results.

The requested IDE screenshot could not be acquired through the desktop app-opening permission workflow during this run. The report therefore labels actual diagnostic text as text evidence, preserves the real CODESYS projects and logs, and does not manufacture a GUI screenshot. This presentation limitation does not replace or weaken the compiler checks, but remains an explicitly incomplete requested artifact. A manual screenshot can be added by opening a preserved project in the installed IDE and rebuilding it.

11 Reproduction and deliverable checks

The source package retains task builders, specifications, manual tests, configuration, agent adapters, CODESYS scripts and the manifest-selected raw records. Source and result hashes allow independent checks; historical absolute paths are resolved against the extracted project root where supported. Reproduction requires an installed CODESYS profile/template and private API credentials. Commands are documented for PowerShell, with `scripts/run.ps1` selecting Project1.

The offline suite passes 38 tests covering dataset validation, status accounting, bounded repair flow, actual baseline integration with controlled transport doubles, evidence hashing/path containment, scalar adapter construction, scan isolation and Python reference consistency. Test doubles live in temporary unit-test directories and never enter benchmark totals. The actual valid–invalid–repaired compiler preflight and actual simulator integration checks provide separate external-system evidence.

The final analysis is reproducible with `scripts/analyze_results.py`; the release audit verifies compiler evidence, task/source hashes, denominators, trace evidence, artifact QA and secret exclusion. The final ZIP contains neither the Conda environment nor API credentials. PDF pages and slide previews are visually reviewed, and PPTX object structure is checked for editable text and shapes. A generated explanation video is verified from the actual media duration and decoded fully; its narration is identified as computer generated.

12 Conclusion

We implemented a working ST coding harness connected to the actual CODESYS compiler and a real IDE-simulation test adapter, then compared it with an established tool-using agent using two verified relay routes. The preserved repair examples demonstrate that compiler feedback can lead to a successful build, while the FIFO example shows why build success alone is insufficient. The resulting benchmark is useful because failures, unsupported cases and evidence boundaries remain visible. Extending controlled timing and composite-interface testing, then repeating the campaign, would improve its coverage and statistical reliability.

References

[1] CODESYS, *Starting Scripts from the Command Line*. `https://content.helpme-codesys.com/en/CODESYS%20Scripting/_cds_starting_script_via_command_line.html`.

[2] CODESYS, *ScriptApplication API*. `https://content.helpme-codesys.com/en/ScriptingEngine/ScriptApplication.html`.

[3] CODESYS, *ScriptSystem API*. `https://content.helpme-codesys.com/en/ScriptingEngine/ScriptSystem.html`.

[4] Hugging Face, *smolagents Agent Reference*. `https://huggingface.co/docs/smolagents/en/reference/agents`.